

Exercício A2: Arquitetura CI/CD com Jenkins

Gustavo Gutierrez Champam - RA 223645

Gustavo Oliveira Camargo - RA 236024

Atividade preenchida em 06/10/2026

Repositórios utilizados: professora integracao_v1; AC1 da equipe educacao-continuada-gamificada; Jogo Enigma V5.1 da professora. Links e evidências estão nas respectivas práticas.

INTRODUÇÃO – Exercício A2

Integração Contínua, Entrega Contínua e Pipeline as Code com Jenkins.

Na Componente Devops & QA, este exercício dá continuidade ao Exercício A1. No A1, os testes automatizados, JUnit, JaCoCo e PMD foram estudados como mecanismos de Quality Assurance. Na A2, essas verificações passam a fazer parte de uma arquitetura automatizada de CI/CD executada pelo Jenkins.

- 1) Integração Contínua (CI):** automatiza a obtenção do código, build, testes e verificações de qualidade, fornecendo feedback rápido a cada alteração.
- 2) Entrega/Implantação Contínua (CD):** evolui a esteira para a criação de containers, disponibilização da aplicação em ambiente de homologação e validação do sistema implantado.
- 3) Pipeline as Code:** permite que a definição da esteira seja versionada no Git por meio do Jenkinsfile, tornando o processo rastreável e reproduzível.
- 4) Observabilidade:** com Prometheus e Grafana permite acompanhar a aplicação após o deploy, conectando operação, qualidade e arquitetura.

OBJETIVO DE APRENDIZAGEM

Evoluir de um Job Jenkins configurado manualmente em modo Freestyle para uma arquitetura CI/CD versionada em Jenkinsfile, com build, testes, qualidade, containers, deploy em homologação, testes E2E e observabilidade.

ARQUITETURA DA A2

```
Git → Jenkins → CI [Build | JUnit | JaCoCo | PMD] → CD [Docker | HOMOL | Health Check | Cypress] → Prometheus/Grafana
```

IMPORTANTE: neste laboratório, o Jenkins está sendo executado na porta 9090. Como essa porta é normalmente utilizada pelo Prometheus, a porta externa do Prometheus será 9091. A API permanece em 8080 e o Grafana em 3001.

```
Jenkins: localhost:9090 | API: localhost:8080 | Prometheus: localhost:9091 | Grafana: localhost:3001
```

PRÁTICA 1 – JENKINS FREESTYLE – INTEGRAÇÃO CONTÍNUA

TEMA: Conhecendo Jenkins, CI e evidências automatizadas de QA

RECURSOS: apresentação Pratica_1

(Ac2)_Jenkins_Projeto_Integracao_CI_Junit_Jacoco_Plugins.pptx, aplicação integracao_v1, Jenkins, Git, JDK 17, Maven, JUnit e JaCoCo.

ROTEIRO DA PRÁTICA

Básico de Aprendizado

- 1) Inicie o Jenkins conforme a apresentação e valide o acesso pelo navegador.
- 2) Confira os plugins Git, Maven Integration, JUnit e JaCoCo.
- 3) Crie um novo Job do tipo Freestyle. Sugestão: A2_P1_integracao_ci.
- 4) Em Gerenciamento de Código Fonte, configure o repositório Git da aplicação integracao_v1.
- 5) Em Build, configure o Maven para executar clean test ou clean verify.
- 6) Em Ações de pós-build, publique os resultados do JUnit.

PUBLICAÇÃO DOS RESULTADOS DE TESTE

No pós-build do Jenkins, utilize o padrão abaixo para localizar os XMLs gerados pelo Maven Surefire:

```
*/target/surefire-reports/*.xml
```

- 7) Configure a publicação da cobertura do JaCoCo.
- 8) Execute o Job e acompanhe o Console Output até o resultado final.

EXPERIMENTO: FALHA → CORREÇÃO

- 1) Faça uma alteração controlada para provocar a falha de um teste.
- 2) Execute novamente o Job e tire um print do build com falha e do trecho relevante do Console Output.
- 3) Corrija a implementação/teste e execute novamente.
- 4) Registre o novo build verde e compare as evidências do JUnit e JaCoCo.

ANÁLISE DA PRÁTICA 1

- Em qual etapa desta prática ocorre CI?

Resposta da equipe: A CI ocorre no checkout do Git e na execução automatizada do Maven, que compila e executa testes e verificações. Utilizamos clean verify tanto no integracao_v1 quanto no projeto AC1 da equipe.

- Por que o Jenkins não substitui Maven, JUnit ou JaCoCo?

Resposta da equipe: Jenkins orquestra e registra a execução. Maven resolve dependências e coordena o build; JUnit executa os testes; JaCoCo mede a cobertura do código exercitado. Cada ferramenta mantém sua responsabilidade.

- O que o build vermelho informa à equipe antes de qualquer deploy?

Resposta da equipe: O vermelho indica que uma etapa falhou e exige investigação antes de promover o artefato. No experimento, a regra de frequência foi alterada de 75 para 76 e dois testes detectaram a regressão.

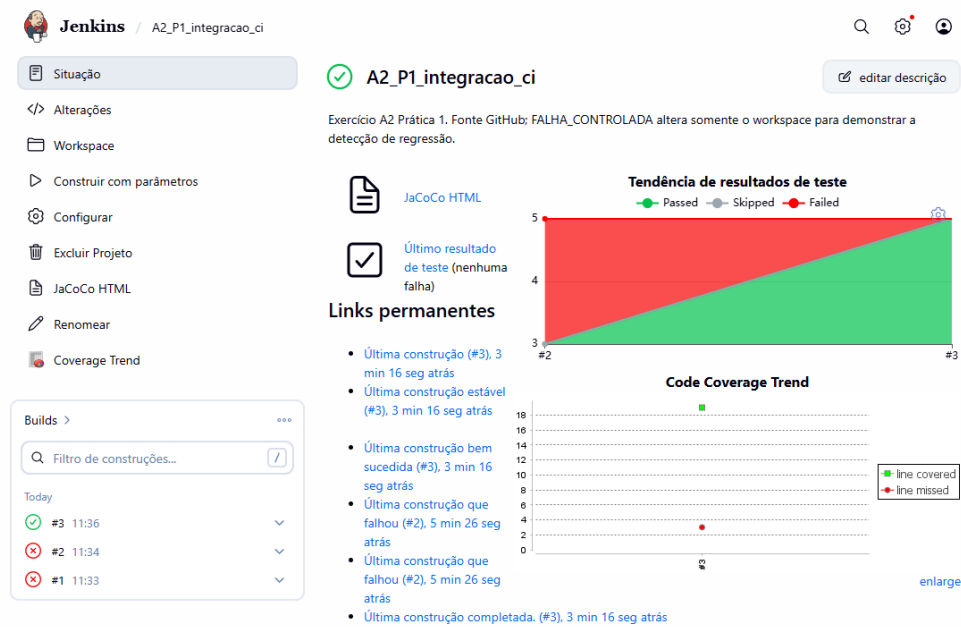
- Qual a vantagem de publicar JUnit e JaCoCo no próprio Job?

Resposta da equipe: Os resultados ficam associados ao histórico do build, facilitando localizar falhas e comparar cobertura. No integracao_v1 publicamos JUnit e JaCoCo; repetimos a integração com os relatórios próprios da AC1.

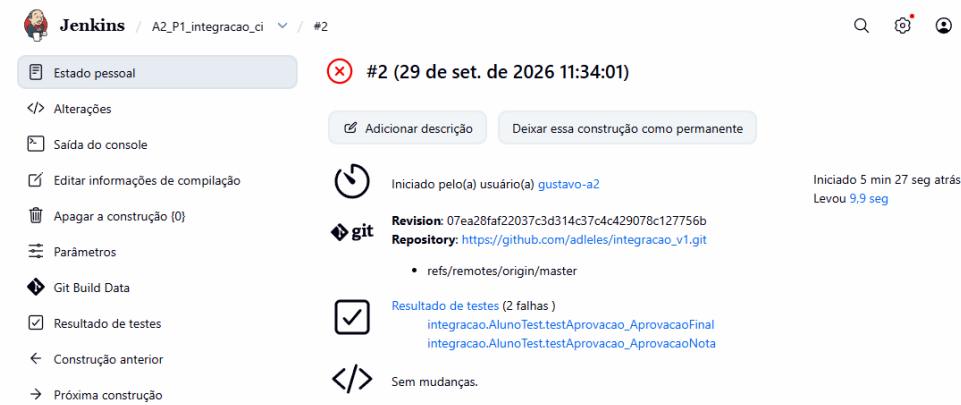
EVIDÊNCIAS DA PRÁTICA 1 — GIT DA PROFESSORA

Fonte: https://github.com/adleles/integracao_v1
Commit: 07ea28faf22037c3d314c37c4c429078c127756b. Jenkins local em 29/09/2026; job Freestyle A2_P1_integracao_ci, JDK 17, Maven clean verify. Plugins Git, Maven Integration, JUnit e JaCoCo.

Job Freestyle integracao_v1 no Jenkins



Build 2 com falha controlada — 5 testes, 2 falhas



FALHA E CORREÇÃO — INTEGRACAO_V1

A alteração controlada ocorreu apenas no workspace. Ao desativar FALHA_CONTROLADA, o checkout limpo restaurou o limite original de frequência. O build 3 terminou em SUCCESS, com 5 testes aprovados. O primeiro build do histórico foi uma falha de configuração e não integra o experimento.

Console da falha controlada

Jenkins / A2_P1_integracao_ci / #2

```
[INFO]
[ERROR] Failures:
[ERROR] AlunoTest.testAprovacao_AprovacaoFinal:46 expected: <true> but was: <false>
[ERROR] AlunoTest.testAprovacao_AprovacaoNota:35 expected: <true> but was: <false>
[INFO]
[ERROR] Tests run: 5, Failures: 2, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 2.644 s
[INFO] Finished at: 2026-09-29T11:34:11-03:00
[INFO] -----
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-
plugin:3.5.4:test (default-test) on project integracao_v1: There are test failures.
[ERROR]
[ERROR] See C:\Users\Gustavo\Documents\Codex\2026-09-29\https-github-com-adeles-
integracao-v1\work\jenkins\home\workspace\A2_P1_integracao_ci\target\surefire-reports
for the individual test results.
[ERROR] See dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and
[date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the
following articles:
```

Build 3 corrigido

Jenkins / A2_P1_integracao_ci / #3

Estado pessoal

Alterações

Saída do console

Editar informações de compilação

Apagar a construção (0)

Parâmetros

Git Build Data

Resultado de testes

Coverage Report

JaCoCo HTML

Construção anterior

#3 (29 de set. de 2026 11:36:11)

Adicionar descrição

Deixar essa construção como permanente

Iniciado pelo(a) usuário(a) [gustavo-a2](#)

Iniciado 3 min 19 seg atrás
Levou 9 seg

Revision: 07ea28faf22037c3d314c37c4c429078c127756b

Repository: https://github.com/adeles/integracao_v1.git

refs/remotes/origin/master

Resultado de testes (nenhuma falha)

Jacoco - Overall Coverage Summary

INSTRUCTION	87%	
BRANCH	100%	
COMPLEXITY	77%	
LINE	86%	
METHOD	67%	
CLASS	100%	

Sem mudanças.

REST API Jenkins 2.541.3

RELATÓRIOS JUNIT E JACOCO — INTEGRACAO_V1

JUnit: 5 aprovados, 0 falhas, 0 erros e 0 ignorados. JaCoCo: 86,36% de linhas e 100% de ramos. No build com falha, a execução interrompeu antes de verify; a cobertura exibida corresponde ao build corrigido.

JUnit publicado no job da aplicação da professora

Estado pessoal

Alterações

Saída do console

Editar informações de compilação

Apagar a construção (0)

Parâmetros

Git Build Data

Resultado de testes

Coverage Report

JaCoCo HTML

Construção anterior

Jenkins / A2_P1_integracao_ci / #3 / Resultado de teste

Resultado de teste 5

5

2

Levou 68 ms.

All tests are passing!

Nice one! All 5 tests are passing.

Todos os testes

Pacote	Falha	Pular	Passed	Total	Duração
integracao	0 -2	0	5 +2	5	35 ms

REST API Jenkins 2.541.3

JaCoCo publicado no job da aplicação da professora

Estado pessoal

Alterações

Saída do console

Editar informações de compilação

Apagar a construção (0)

Parâmetros

Git Build Data

Resultado de testes

Coverage Report

JaCoCo HTML

Construção anterior

Jenkins / A2_P1_integracao_ci / #3 / Jacoco

JaCoCo Coverage Report

Download jacoco.exec binary coverage file

18

16

14

12

10

8

6

4

2

0

line covered

line missed

Overall Coverage Summary

name	instruction	branch	complexity	line	method	class
all classes	87% M: 9 C: 58	100% M: 0 C: 8	77% M: 3 C: 10	86% M: 3 C: 19	67% M: 3 C: 6	100% M: 0 C: 1

Coverage Breakdown by Package

name	instruction	branch	complexity	line	method	class
integracao	87% M: 9 C: 58	100% M: 0 C: 8	77% M: 3 C: 10	86% M: 3 C: 19	67% M: 3 C: 6	100% M: 0 C: 1

REST API Jenkins 2.541.3

INTEGRAÇÃO DA AC1 — GIT PESSOAL DA EQUIPE

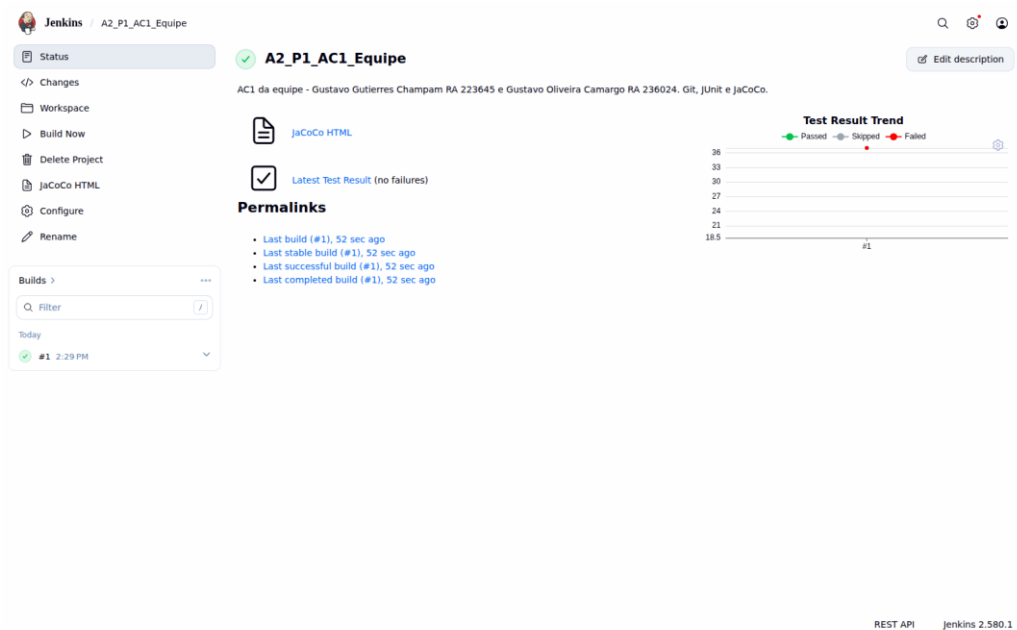
Fonte: <https://github.com/Gustavo-Champam/educacao-continuada-gamificada>

Commit executado: 5095ff220d9e31f03622ed417023aa6283052a44

Execução Jenkins em 06/10/2026: <https://github.com/Gustavo-Champam/atividade-a2-jenkins-cicd/actions/runs/37479149544>

Criamos o job Freestyle A2_P1_AC1_Equipe com checkout do Git pessoal, JDK 17 e mvn -B clean verify. Publicamos os XMLs JUnit do Surefire e o relatório HTML JaCoCo no Jenkins. A execução foi feita no mesmo tipo de laboratório Linux remoto, sem depender do Docker Desktop do PC.

Job Freestyle da AC1 com resultado SUCCESS

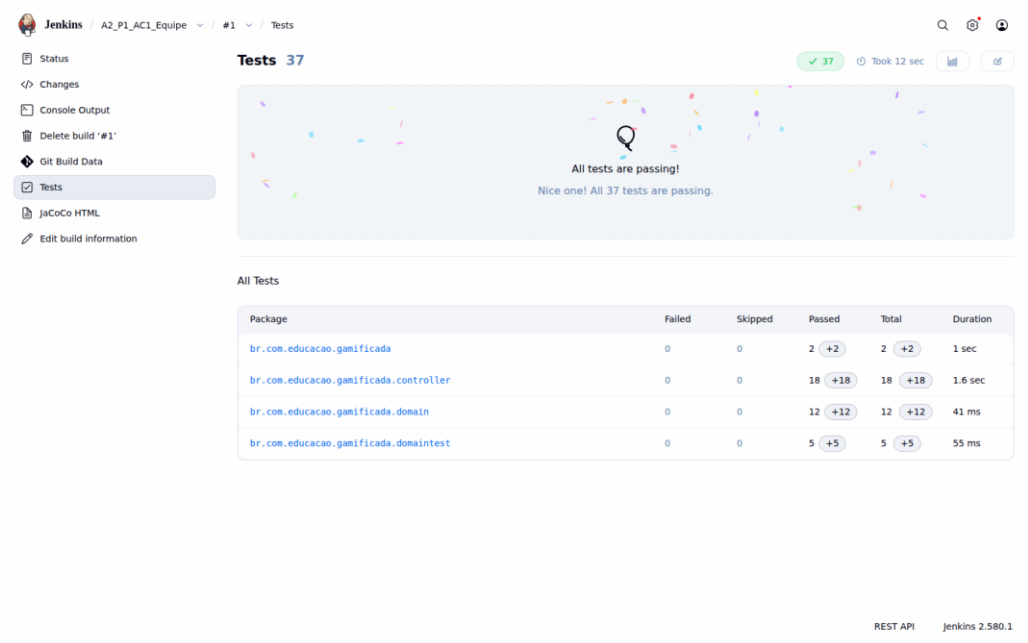


O teste condicional `AlunoPostgresTest` foi habilitado com `TESTAR_POSTGRES=true` e banco PostgreSQL 17 real. Resultado: 37 testes aprovados, nenhuma falha, erro ou teste ignorado. Isso inclui as verificações locais com H2 e a persistência no PostgreSQL.

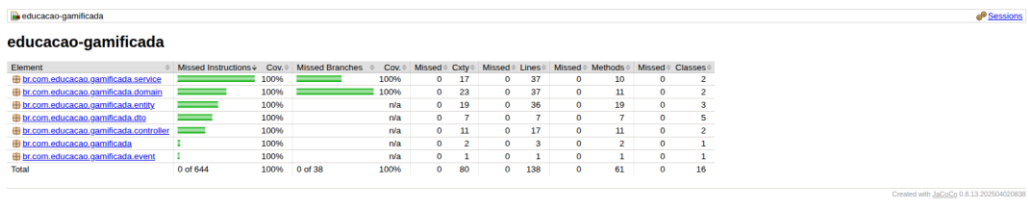
RELATÓRIOS JUNIT E JACOCO — AC1 DA EQUIPE

JUnit: 37 aprovados. JaCoCo: 138/138 linhas e 38/38 ramos cobertos (100%); 644/644 instruções, 61/61 métodos e 16/16 classes cobertos. O gate de cobertura do pom.xml foi aprovado. Esses números são próprios da AC1, não do integracao_v1 ou do Jogo Enigma.

Relatório JUnit da AC1 publicado no Jenkins



Relatório JaCoCo real da AC1



PRÁTICA 2 – PIPELINE AS CODE COM JENKINSFILE

TEMA: Evolução do Freestyle para Pipeline versionado no Git

RECURSOS: aplicação jogo-enigma-api-v1-A2-corrigido, Jenkins Pipeline, Git, Maven, JUnit, JaCoCo, PMD e Docker Desktop.

ROTEIRO DA PRÁTICA

- 1) Abra o projeto jogo-enigma e localize o arquivo Jenkinsfile na raiz.
- 2) No Jenkins, crie um novo item do tipo Pipeline. Sugestão:
A2_P2_jogo_enigma_pipeline.
- 3) Em Pipeline, selecione “Pipeline script from SCM”. Configure SCM = Git e informe o repositório da equipe.
- 4) Configure Script Path = Jenkinsfile.
- 5) Execute o pipeline e acompanhe a execução pelo Stage View e pelo Console Output.
- 6) Identifique no Jenkinsfile quais estágios pertencem à CI e quais iniciam a CD.
IMPORTANTE: o mesmo Jenkinsfile será reutilizado na Prática 3; não será criado um segundo pipeline.

LEITURA ARQUITETURAL DOS STAGES

Stage	Classificação	Responsabilidade
Checkout	Base	Obter a versão rastreável do código
JUnit + JaCoCo	CI	Executar testes e registrar cobertura
PMD	CI	Executar análise estática
Package	CI	Gerar o artefato da aplicação
Docker Build	CI → CD	Gerar unidade implantável
Deploy HOMOL	CD	Subir o ambiente integrado
Health Check	CD	Validar disponibilidade da API
Cypress E2E	CD	Validar o fluxo ponta a ponta

ANÁLISE DA PRÁTICA 2

- Qual a principal diferença entre configurar a esteira no Freestyle e defini-la em Jenkinsfile?

Resposta da equipe: Freestyle guarda os comandos e publicadores na configuração do job. Pipeline as Code descreve a sequência e as condições em um Jenkinsfile versionado. O job das Práticas 2 e 3 carrega esse arquivo diretamente do Git.

- Por que versionar o pipeline junto ao código?

Resposta da equipe: O Git permite revisar, comparar e reverter mudanças na esteira junto com a aplicação. O commit identifica tanto o código executado quanto a definição do processo, melhorando rastreabilidade e reprodução.

- Se o estágio de testes falhar, os estágios de container/deploy devem continuar? Justifique.

Resposta da equipe: Não. O retorno de falha do Maven interrompe o fluxo e impede os estágios posteriores. A publicação dos resultados continua nas ações de finalização, preservando o diagnóstico sem mascarar o erro.

- Qual estágio representa a transição mais clara entre CI e CD?

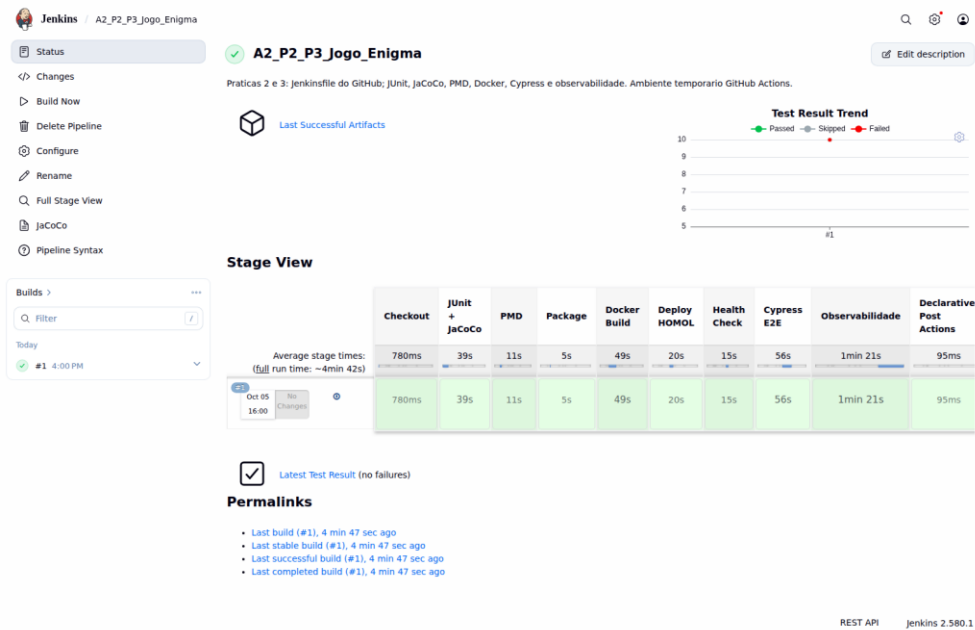
Resposta da equipe: Docker Build é a transição CI para CD, pois transforma o artefato validado em imagem implantável. Deploy HOMOL marca a disponibilização efetiva no ambiente; Health Check e Cypress validam a implantação.

EVIDÊNCIAS DA PRÁTICA 2 — JOGO ENIGMA V5.1

Base da professora: <https://github.com/adleles/jogo-enigma-api-v1-A2-portavel-v5.1>
Repositório da execução: <https://github.com/Gustavo-Champam/atividade-a2-jenkins-cicd>
Execução aprovada em 05/10/2026: <https://github.com/Gustavo-Champam/atividade-a2-jenkins-cicd/actions/runs/37337295060>

A versão V5.1 foi confirmada no Git da professora. O mesmo Jenkinsfile atende às Práticas 2 e 3. Configuração: Pipeline script from SCM, Git, branch main, Script Path Jenkinsfile. Commit executado: 24ca6ff6ff5e186461d386f050d51f48c9d79509.

Stage View real — todos os estágios concluídos com sucesso



JUnit: 7 testes Java aprovados, incluindo PostgreSQL real via Testcontainers; Cypress: 3 cenários aprovados. JaCoCo da API: linhas 83,33%, ramos 50% e instruções 48,48%. PMD foi executado e arquivado, com 0 alertas. A política original gera relatório com pmd:pmd, sem gate pmd:check.

PRÁTICA 3 – CD, CONTAINERS E OBSERVABILIDADE

TEMA: Deploy em HOMOL, validação pós-deploy e monitoramento

Nesta prática será utilizado o mesmo projeto e o mesmo Jenkinsfile da Prática 2. O foco agora deixa de ser a leitura do Pipeline as Code e passa a ser a arquitetura de CD e observabilidade. Assim, CI e CD fazem parte de uma única esteira; o que muda entre as práticas é o objeto de aprendizagem.

ARQUITETURA DO AMBIENTE

```
Jenkins → Docker Build → Docker Compose HOMOL → API + BFF + PostgreSQL  
→ Health Check → Cypress → Prometheus → Grafana
```

- 1) Com o Docker Desktop em execução, rode o pipeline até o estágio de deploy em HOMOL.
- 2) Valide a API pelo endpoint `http://localhost:8080/actuator/health`.
- 3) Valide o BFF/Front em `http://localhost:3000`.
- 4) Acesse o Prometheus em `http://localhost:9091`. Em Status → Target health, verifique o target `jogo-enigma-api`. O endpoint esperado é `http://api:8080/actuator/prometheus` e o estado deve ser UP.
- 5) Acesse o Grafana em `http://localhost:3001` (admin/admin). Em Connections → Data sources, valide o Prometheus. Dentro da rede Docker, a URL deve permanecer `http://prometheus:9090`. Depois, em Explore, execute a consulta `up{job="jogo-enigma-api"}`; o valor 1 indica que o target está disponível.
- 6) Observe a execução do Cypress após o deploy e explique por que o teste E2E ocorre nessa posição da esteira.

RELAÇÃO ENTRE PROMETHEUS E GRAFANA

A aplicação Spring Boot expõe métricas em `/actuator/prometheus`. O Prometheus realiza o scrape desse endpoint, coleta e armazena as séries temporais. O Grafana não coleta diretamente da aplicação: ele utiliza o Prometheus como Data Source, executa consultas PromQL e transforma os dados em gráficos, indicadores e dashboards.

Fluxo de observabilidade: Spring Boot → `/actuator/prometheus` → Prometheus → Grafana.

PORTAS DO LABORATÓRIO

Jenkins usa 9090 no host. Por isso, o Docker Compose publica o Prometheus como 9091:9090. A porta 9091 é usada pelo navegador no Windows; entre containers, Grafana e Prometheus continuam se comunicando por `prometheus:9090`.

ANÁLISE DA PRÁTICA 3

- Por que um BUILD SUCCESS não garante que a aplicação implantada esteja operacional?

Resposta da equipe: BUILD SUCCESS confirma somente as verificações configuradas naquele momento. A aplicação pode falhar depois por indisponibilidade do banco, rede, configuração ou recursos. Por isso verificamos saúde, fluxos E2E e métricas após o deploy.

- Qual é a função do Health Check no CD?

Resposta da equipe: Confirmar a prontidão da API implantada. A etapa faz tentativas limitadas em /actuator/health e exige resposta HTTP válida com status UP. Não substitui a verificação de um fluxo de negócio.

- Qual é o papel do Docker Compose no ambiente de homologação?

Resposta da equipe: Descrever e iniciar API, BFF, PostgreSQL, Prometheus e Grafana, com redes, volumes, configurações e portas. O banco deve estar saudável antes da API; iniciar containers, sozinho, não comprova que o sistema está pronto.

- Qual é a relação entre Prometheus e Grafana? Explique por que o Prometheus coleta/armazena métricas e o Grafana consulta essas métricas para visualização.

Resposta da equipe: Prometheus coleta /actuator/prometheus e armazena séries temporais. Grafana consulta o Prometheus por PromQL para exibir gráficos e indicadores. Na rede Docker, o datasource usa http://prometheus:9090; no navegador, o Prometheus é acessado em 9091.

EVIDÊNCIAS DA PRÁTICA 3 — DEPLOY HOMOL

Jenkins e Docker foram executados em runner Linux temporário do GitHub Actions. Actions prepara a máquina e coleta evidências; Jenkins executa a esteira. Portas: Jenkins 9090, API 8080, BFF 3000, Prometheus 9091 e Grafana 3001. As URLs localhost das capturas pertencem ao runner, que é descartado ao terminar.

Serviço	Estado	Imagem
api	running	jogo-enigma-api:1
bff	running	a2homol-bff
grafana	running	grafana/grafana:12.1.1
pgadmin	running	dpage/pgadmin4:9
postgres	running healthy	postgres:17-alpine
prometheus	running	prom/prometheus:v3.5.0

Tabela transcrita de docker compose ps --format json, preservado no pacote de evidências. Os seis serviços estavam em execução e o PostgreSQL estava healthy. O health check retornou status UP.

Health check real da API

```

Pretty-print
{"status":"UP","groups":[{"liveness","readiness"]}
```

FRONTEND E TESTES CYPRESS

Frontend servido pelo BFF em homologação

Jogo Enigma

Front → BFF Node.js → API Spring Boot → PostgreSQL

Nome	E-mail	0	Cadastrar
-----------------------------------	-------------------------------------	--------------------------------	--

Participantes

- Ana E2E - ana1791216241190@teste.com - 80 - MESTRE
- Dup - dup1791216242628@teste.com - 10 - INICIANTE

Cypress — cadastro ponta a ponta com classificação MESTRE

Jogo Enigma

Front → BFF Node.js → API Spring Boot → PostgreSQL

Nome	E-mail	0	Cadastrar
-----------------------------------	-------------------------------------	--------------------------------	--

Cadastrado com sucesso

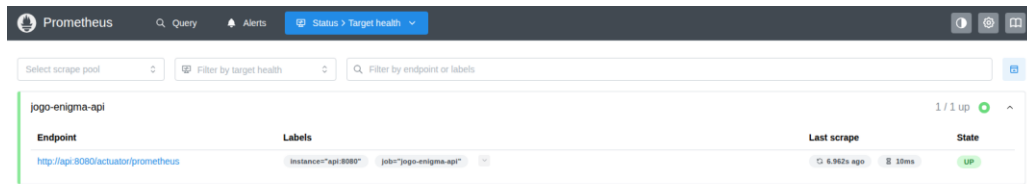
Participantes

- Ana E2E - ana1791216241190@teste.com - 80 - MESTRE

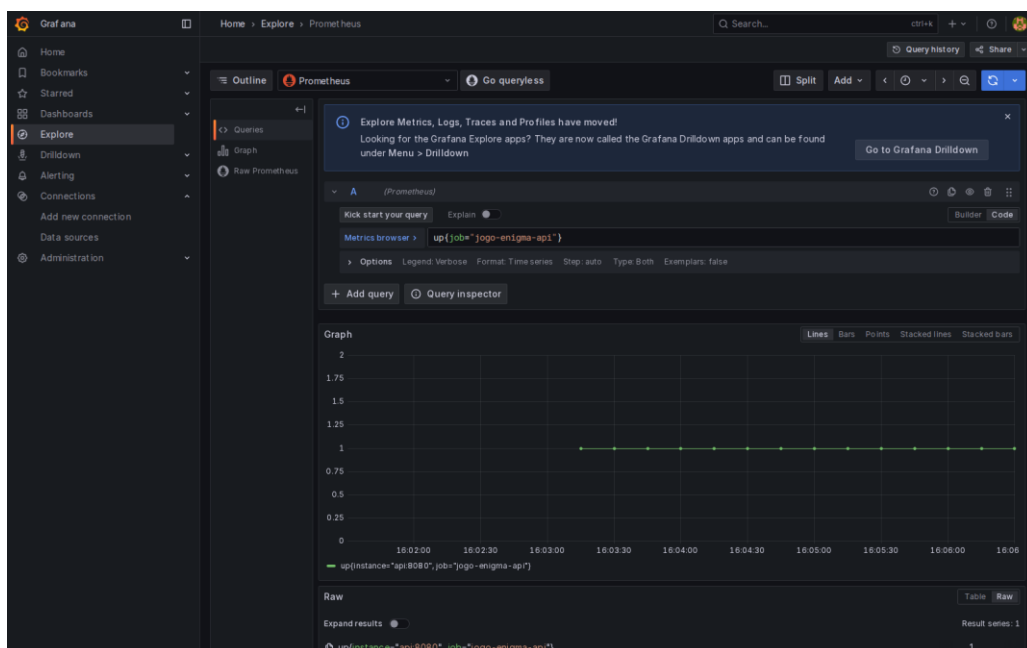
Três cenários aprovados: abertura da home; cadastro de participante; rejeição de e-mail duplicado com HTTP 400. Cypress ocorre depois do deploy porque precisa do ambiente integrado disponível. XMLs, capturas e vídeo da execução foram preservados.

PROMETHEUS E GRAFANA — COLETA E CONSULTA

Prometheus — *jogo-enigma-api UP, endpoint api:8080/actuator/prometheus*



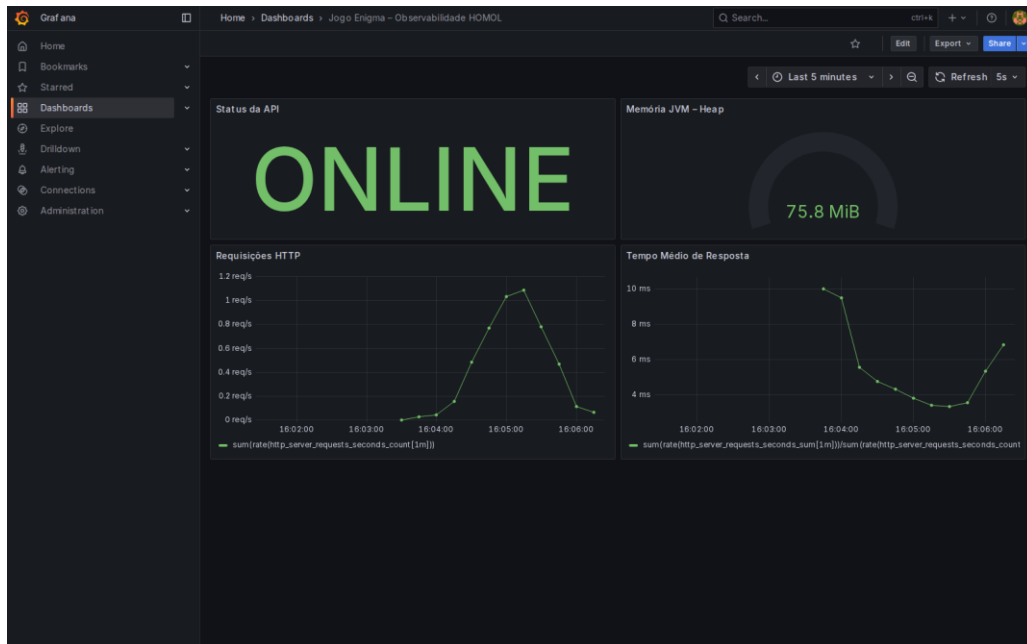
Grafana Explore — *up{job="jogo-enigma-api"} retorna 1*



O datasource foi validado e usa `http://prometheus:9090`. `up=1` indica sucesso na última coleta; não comprova sozinho todos os fluxos de negócio. Foram usadas senhas temporárias do laboratório em vez de manter `admin/admin`.

DASHBOARD — JOGO ENIGMA OBSERVABILIDADE HOMOL

Quatro painéis com métricas reais



Status da API: `up{job="jogo-enigma-api"}`

Requisições HTTP: `sum(rate(http_server_requests_seconds_count[1m]))`

Tempo Médio de Resposta:

`sum(rate(http_server_requests_seconds_sum[1m]))/sum(rate(http_server_requests_seconds_count[1m]))`

Memória JVM – Heap: `sum(jvm_memory_used_bytes{area="heap"})`

Status: Stat com 1=ONLINE e 0=OFFLINE; requisições e latência: Time series; heap: Gauge em bytes IEC. Acessos reais à API e BFF geraram tráfego. O dashboard foi provisionado por JSON e validado no Grafana.

ENTREGA – EXERCÍCIO A2

EVIDÊNCIAS DA EQUIPE

A equipe deverá entregar um único PDF contendo o link do Git utilizado e as evidências das três práticas. As evidências devem mostrar o processo executado pela própria equipe.

- 1) Prática 1: print do Job Freestyle, build com falha, build corrigido, JUnit e JaCoCo publicados no Jenkins.
- 2) Prática 2: print do Stage View do Pipeline e explicação da diferença entre Freestyle e Pipeline as Code.
- 3) Prática 3: containers em execução, health check da API, Cypress, Prometheus com target jogo-enigma-api em UP e Grafana consultando `up{job="jogo-enigma-api"}` com resultado 1.
- 4) Desenhe o fluxo executado pela equipe e identifique visualmente quais etapas correspondem a CI, CD e Observabilidade.
- 5) Inclua uma conclusão curta: “O que mudou arquiteturalmente do Exercício A1 para o Exercício A2?”

CRITÉRIOS DE AVALIAÇÃO

Critério	Peso	Evidência esperada
Prática 1 – Freestyle e CI	25%	Job funcional; falha/correção; JUnit e JaCoCo
Prática 2 – Jenkinsfile	30%	Pipeline executado e interpretação dos stages
Prática 3 – CD e observabilidade	30%	Deploy HOMOL; health; E2E; Prometheus/Grafana
Análise arquitetural	15%	Fluxo CI/CD coerente e justificativas

FLUXO EXECUTADO PELA EQUIPE

BASE — Git professora + Git pessoal → Jenkins
CI — Maven → JUnit → JaCoCo → PMD → Package
CI → CD — Docker Build da API e do BFF
CD — Compose HOMOL → API + BFF + PostgreSQL → Health Check → Cypress
OBSERVABILIDADE — Actuator → Prometheus → Grafana

O integracao_v1 e a AC1 percorrem a integração Freestyle com JUnit e JaCoCo. O Jogo Enigma percorre a esteira completa CI/CD e observabilidade. Os três repositórios têm evidências e resultados separados.

CONCLUSÃO DA EQUIPE

Na AC1, trabalhamos mecanismos de qualidade e testes. Na A2, esses mecanismos foram integrados a uma arquitetura automatizada e rastreável: Git e Jenkins coordenam testes, cobertura, qualidade, empacotamento, containers e deploy. Health Check e Cypress validam o sistema implantado, enquanto Prometheus e Grafana mostram seu comportamento operacional. Integramos ainda nosso projeto AC1 ao Jenkins, publicando seus próprios relatórios JUnit e JaCoCo.

QUESTÕES PARA FECHAMENTO

- 1) Em que ponto da A2 termina a Integração Contínua e começa a Entrega/Implantação Contínua?

Resposta da equipe: A CI inclui build, testes, cobertura, PMD e Package. Docker Build produz a unidade implantável; a CD se concretiza no Deploy HOMOL e continua com Health Check e Cypress.

- 2) Qual vantagem arquitetural o Jenkinsfile oferece em relação à configuração exclusivamente Freestyle?

Resposta da equipe: O Jenkinsfile torna a orquestração versionada, revisável e reproduzível, mantendo explícitas as dependências entre validação, empacotamento e deploy, em vez de depender somente de configuração manual.

- 3) Por que JUnit deve ser executado antes da criação/deploy do container e Cypress depois do deploy?

Resposta da equipe: JUnit verifica rapidamente unidades e integrações controladas antes da promoção da imagem. Cypress precisa dos serviços implantados para exercitar navegador, BFF, API e banco como um sistema integrado.

- 4) Como observabilidade se relaciona com a qualidade de uma aplicação após sua implantação?

Resposta da equipe: As métricas mostram disponibilidade da coleta, volume de requisições, latência e uso de memória. Complementam os testes ao permitir observar degradações durante a operação, sem provar sozinhas a correção de todos os fluxos.

- 5) Qual conhecimento do Exercício A1 foi reutilizado diretamente na construção desta esteira?

Resposta da equipe: Reutilizamos JUnit, JaCoCo, o ciclo Maven e os mecanismos de QA estudados na AC1. Integramos também o projeto real educacao-continuada-gamificada ao Jenkins, com relatórios próprios e teste de persistência PostgreSQL.

- 6) Faça um Dashboard no Grafana, conforme tutorial Grafana que está no CANVAS.

Resposta da equipe: O dashboard Jogo Enigma – Observabilidade HOMOL foi criado e validado com quatro painéis: Status da API (Stat), Requisições HTTP (Time series), Tempo Médio de Resposta (Time series) e Memória JVM – Heap (Gauge em bytes IEC). A captura está nas evidências da Prática 3.